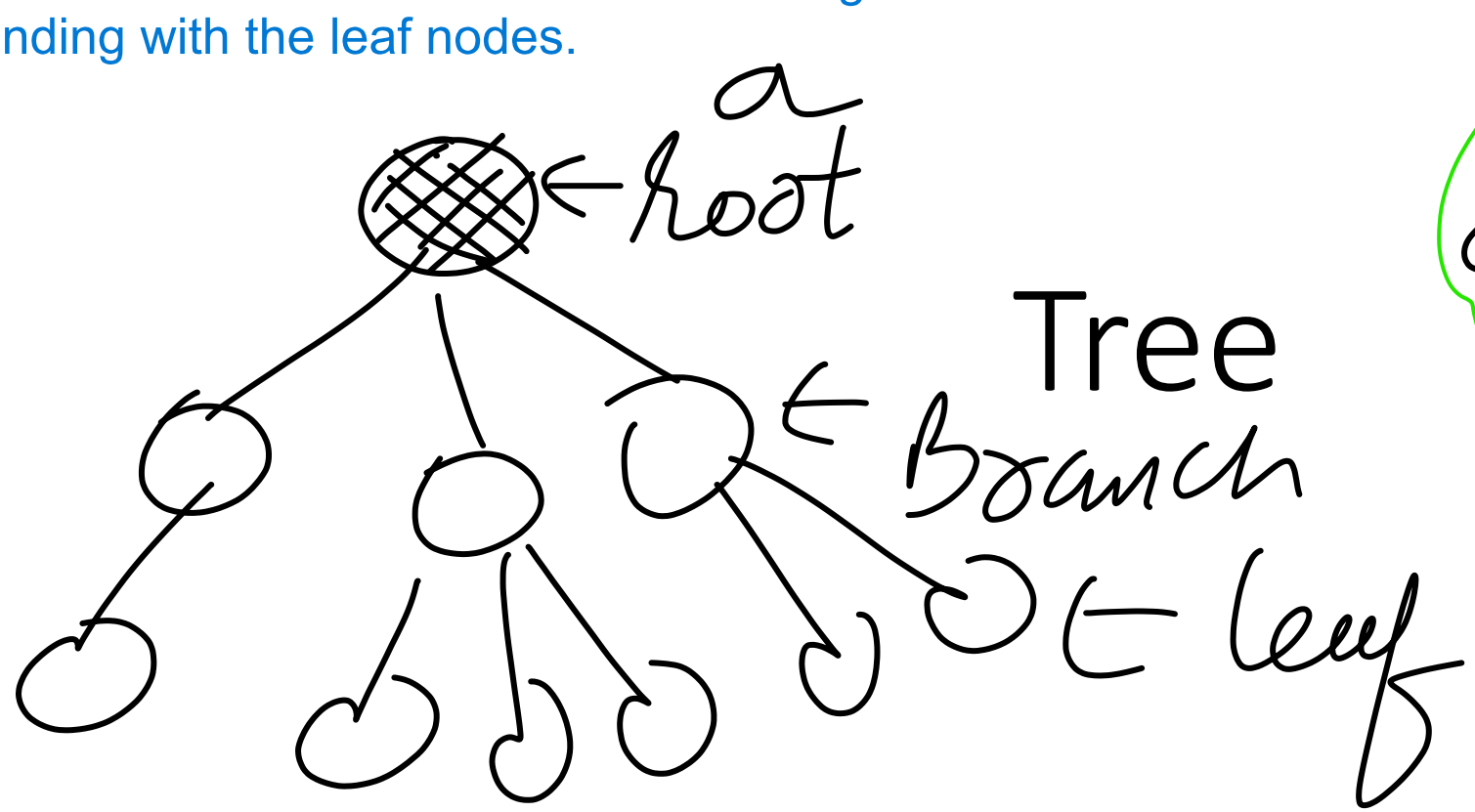
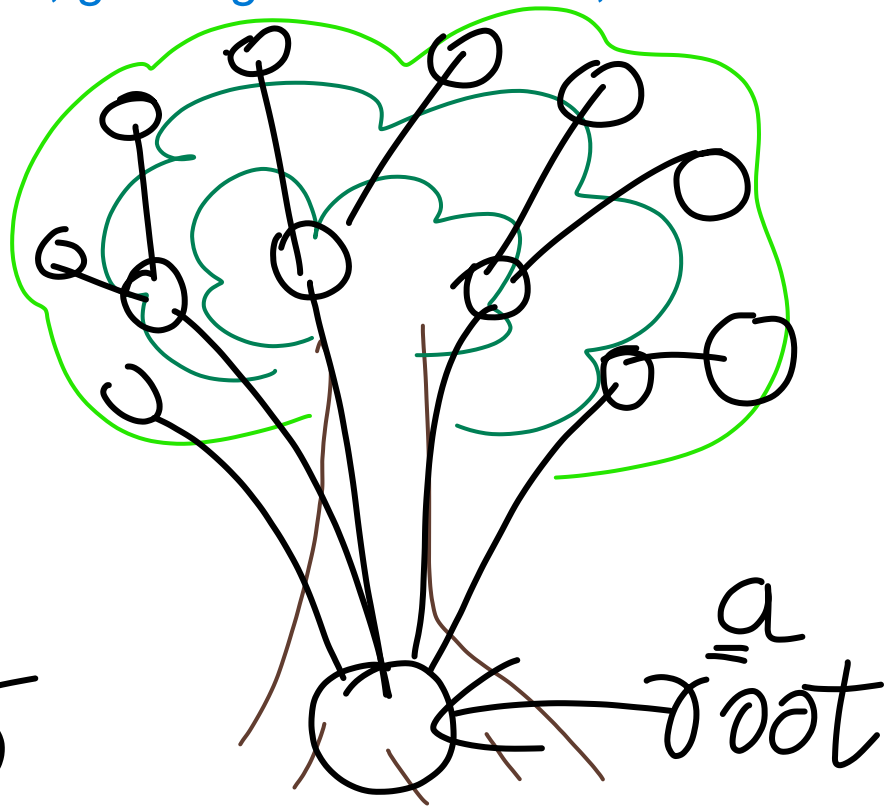
It is a nonlinear hierarchical structure, mostly dynamic in nature but can also be implemented using a static implementation.

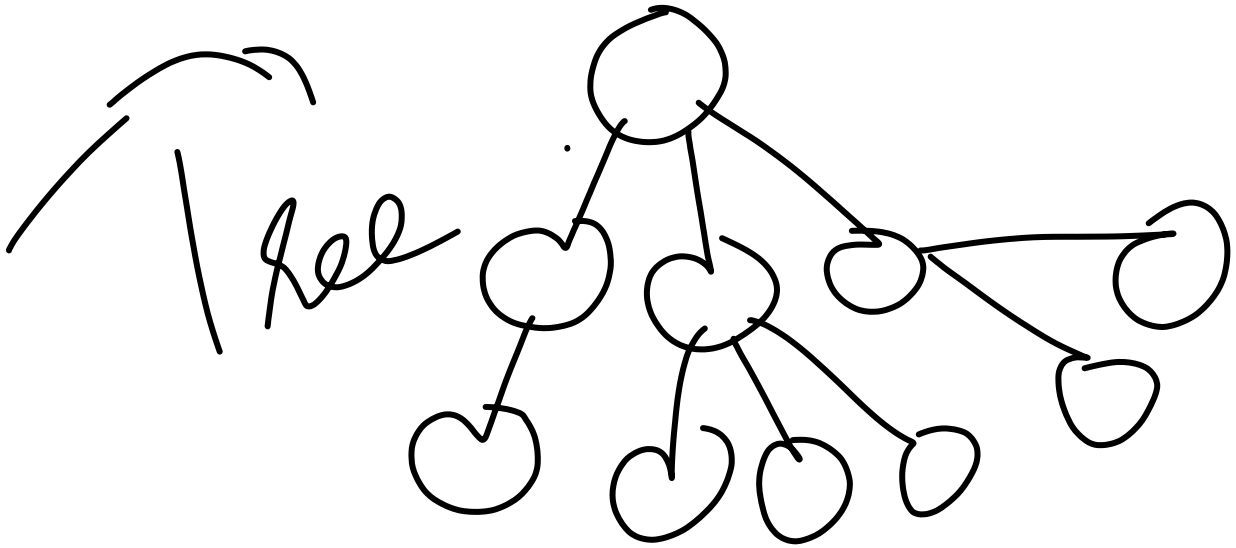
A tree is a hierarchical structure starting with a node called the root, growing with branches, and ending with the leaf nodes.

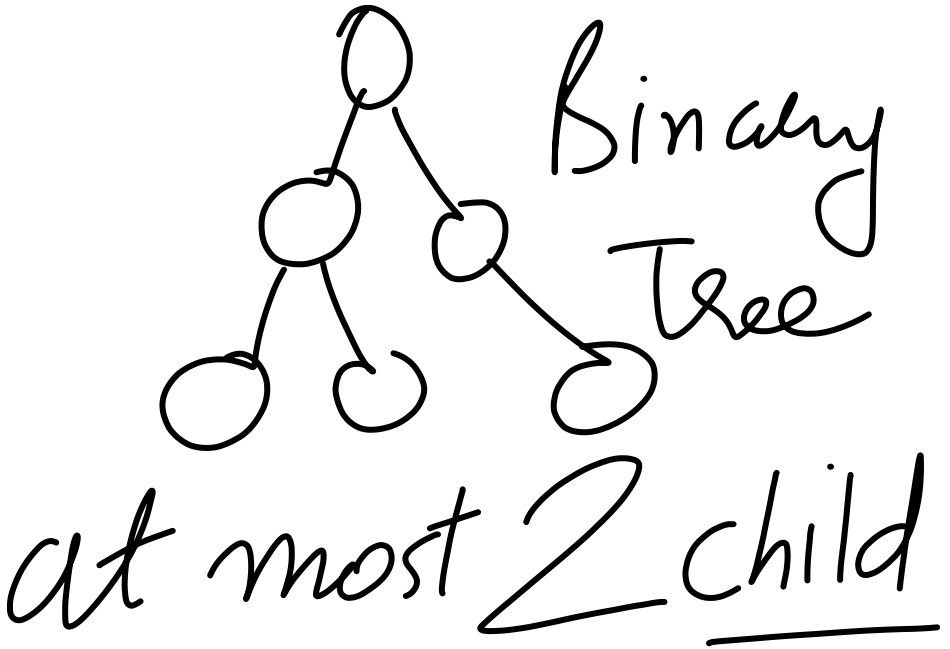




TREE

Tree is very commonly used in applications like
1. Autocomplete, autocorrect
2. Very commonly used for large-scale data sorting and searching
3. Also used in a hierarchical storage of the cloud
4. Also used by AI in algorithms like branch and bound, called a decision tree .
5. Tree is very commonly used for cloud storage, searching functionality, and hierarchical storage on an infra model.





Binary Tree

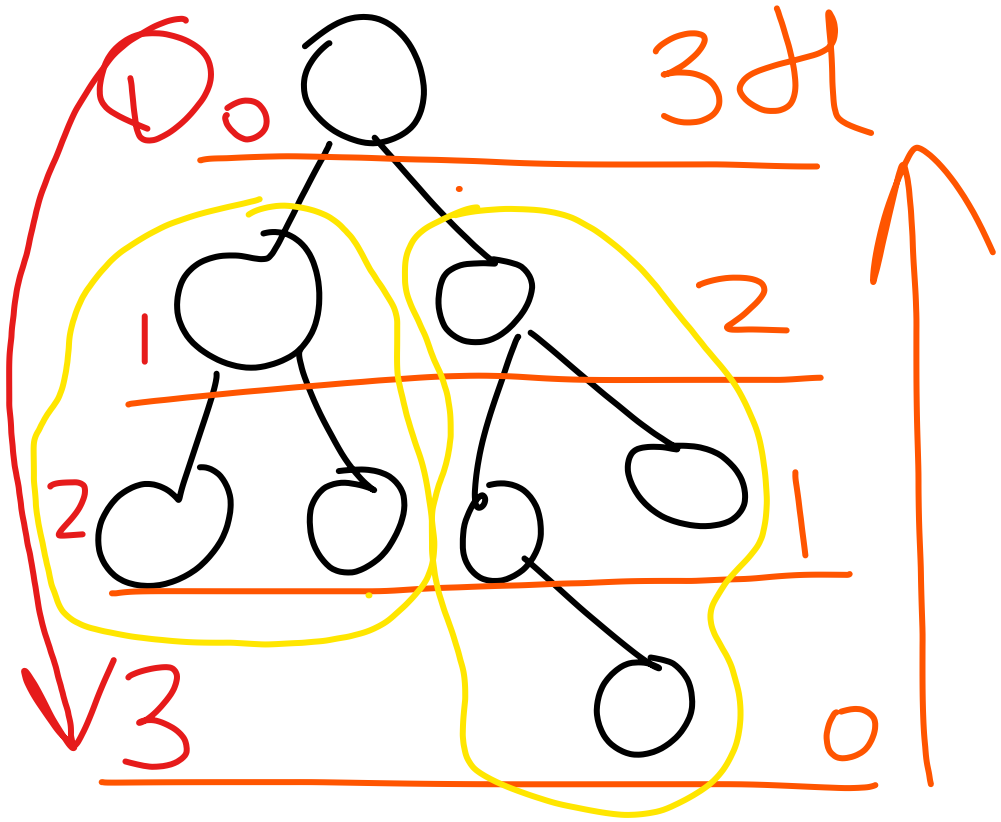
- Terms
  - Depth
  - Height

  Height and depth of a tree is the length of the longest running branch.
  Height is taken from bottom to top and depth is taken from top to bottom
  - Subtree

  Part of a tree that does not involve the root
  - Ancestor

  Root or the subtree root is an ancestor.
  - Leaf node
  - Root node

  A node which does not have any child is considered to be the last node of the tree as of now.
  - Root is the ancestor of all the nodes.
    It is the starting point of the tree.
    It does not have any parent but the tree originates from the root.

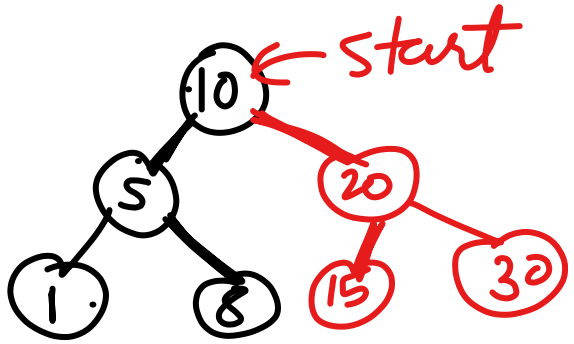


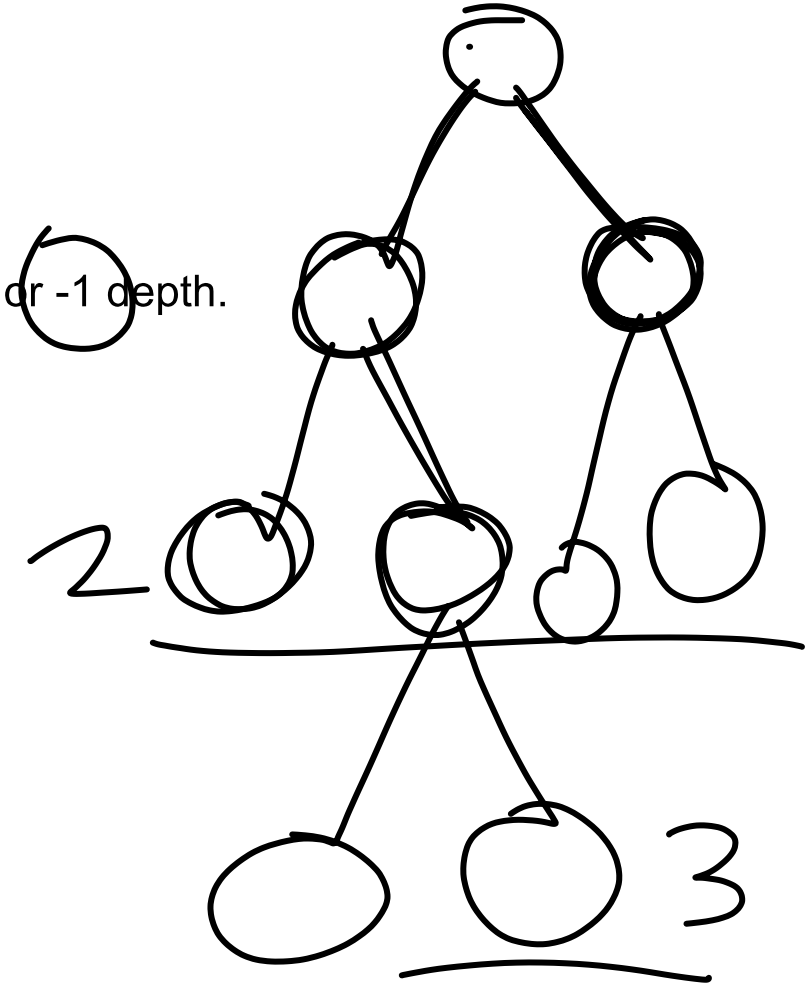
left
Sub
Tree

Types of Binary Tree

- Complete All leaf nodes are at exactly one depth.
- Nearly complete All except one leaf node are either or +1 depth or -1 depth.
- Strict Each node can have either two child nodes or zero.
- Binary Search Tree

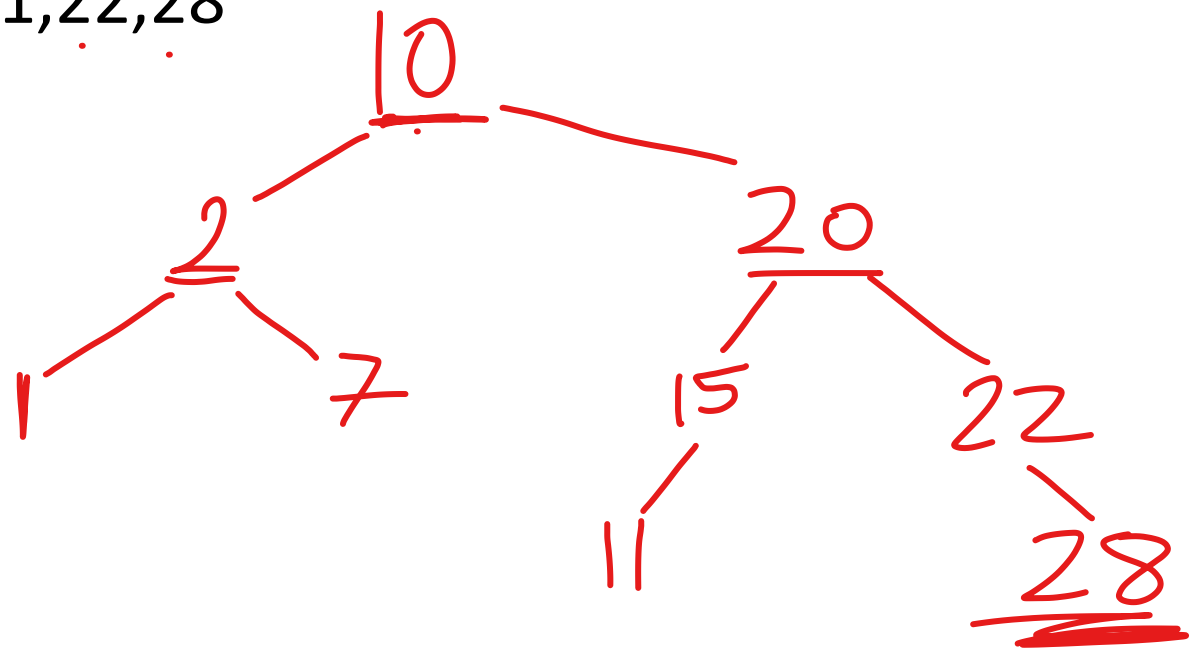
  A specialized version of a binary tree made for searching and sorting
  In this data is arranged such that
  All elements lesser than the parent are to the left.
  All elements greater than or equal to the parent are to the right.





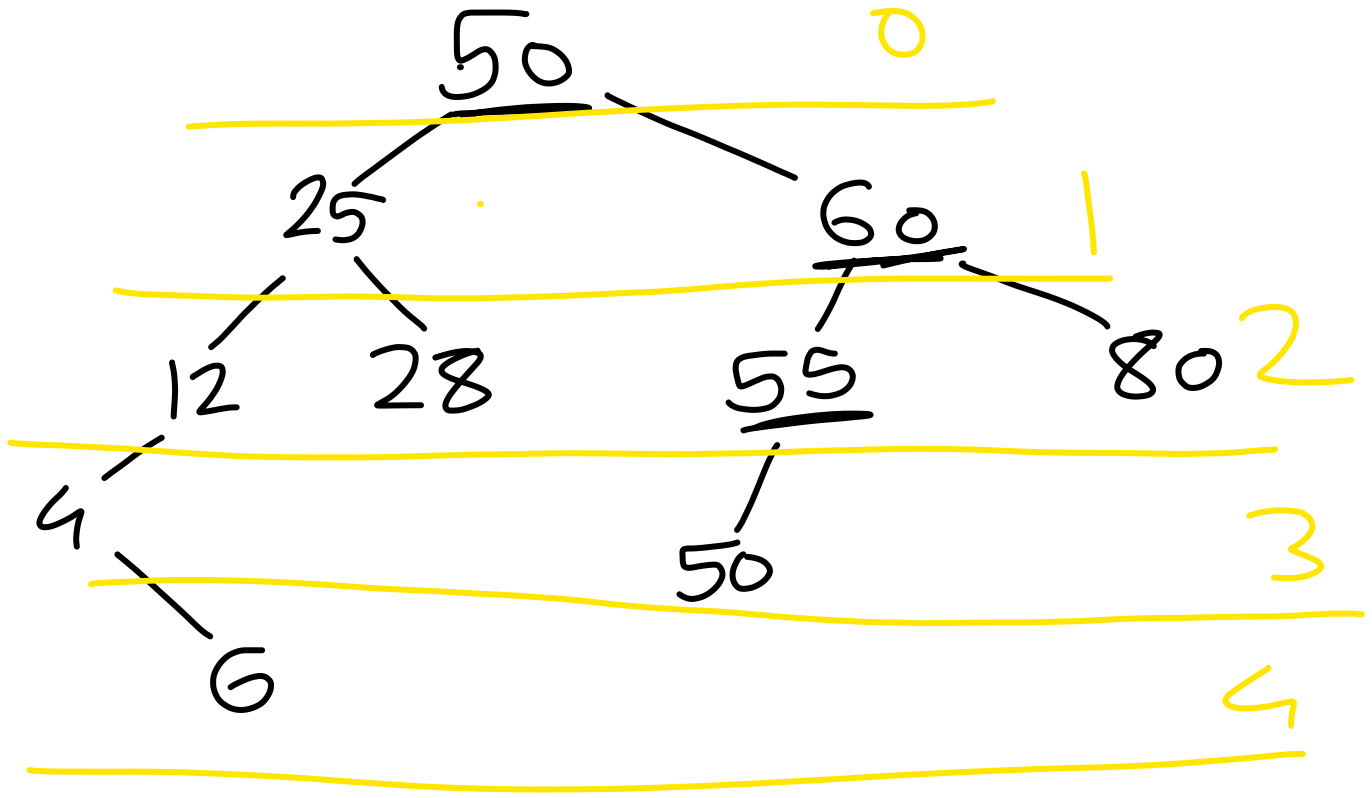
Construct Binary Search Tree

- 10,2,7,1,20,15,11,22,28



Construct Binary Search Tree

- 50,25,12,60,80,55,50,4,6,28



Tree Traversal

Traversal simply refers to the order in which a node or every node is to be visited.

prefix/preorder
+ab

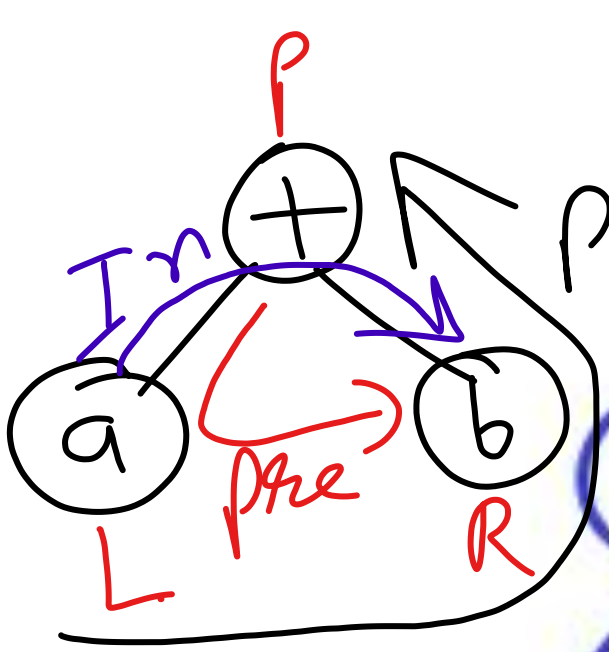
PLR

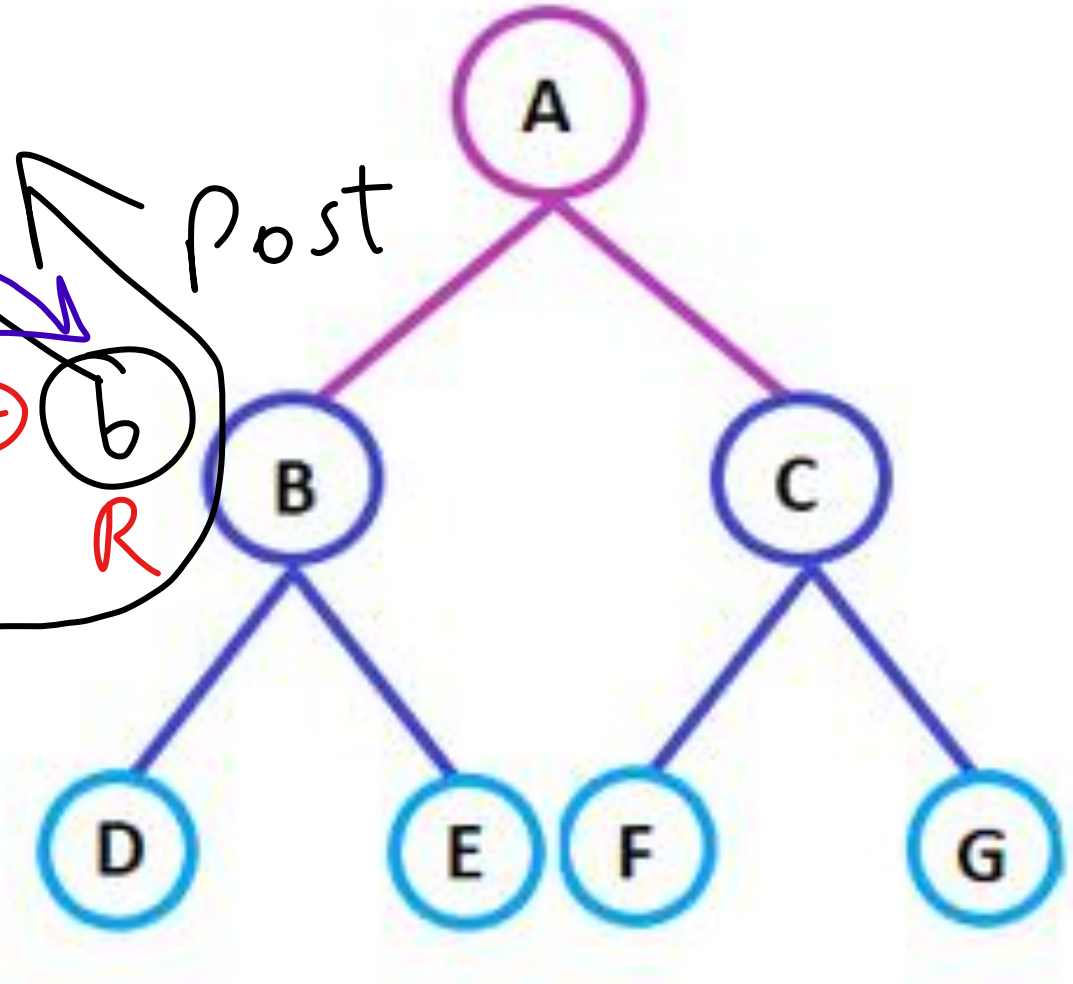
infix/inorder
a+b

LPR

postfix/postorder
ab+

LRP





Construct tree

- Inorder sequence: D B E A F C
- Preorder sequence: A B D E C F

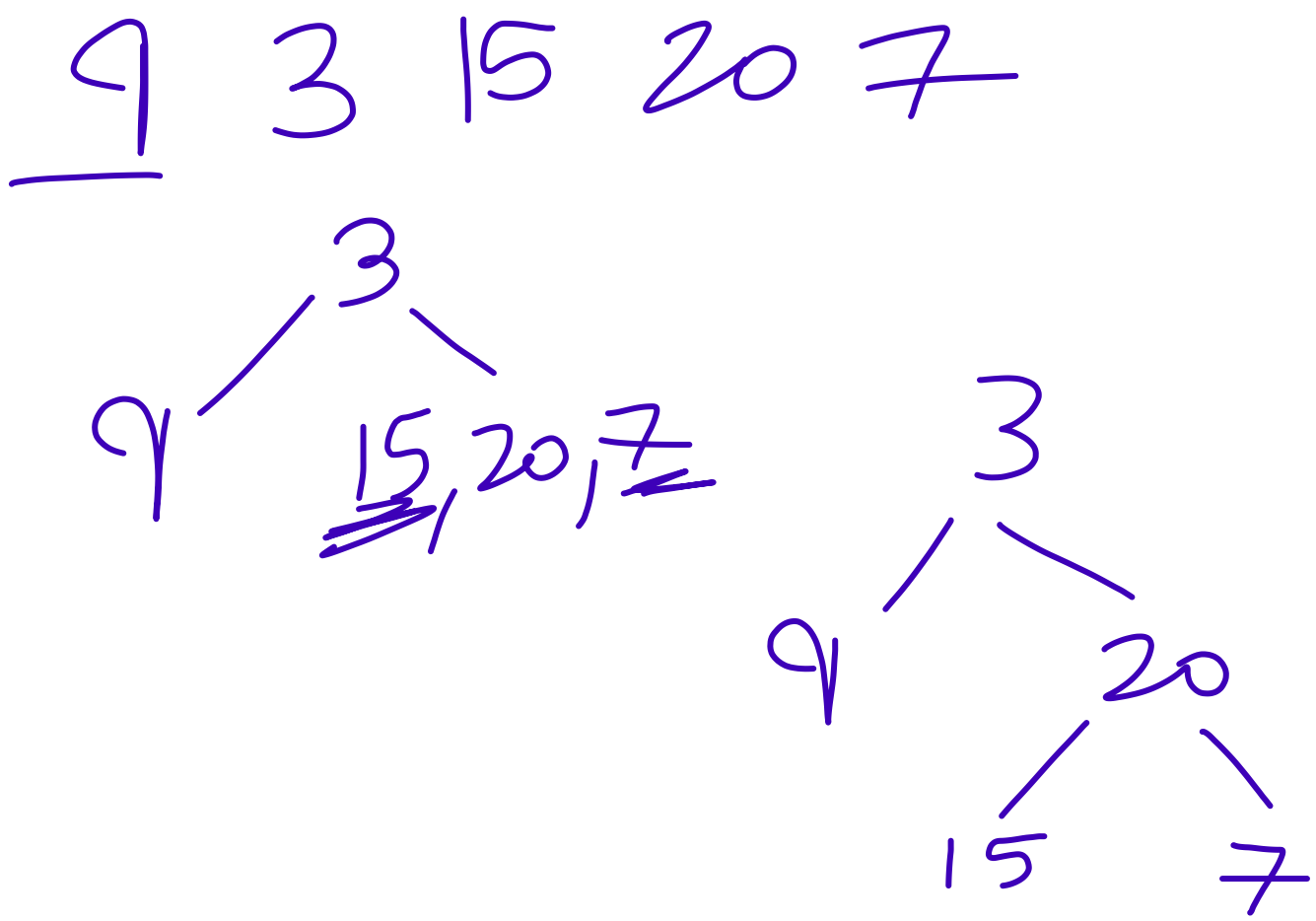
We will use in-order sequence for construction of a tree.
We will follow insertion or breaking sequence using pre-order.

First write down the entire thing in order as it is.
Going from start to end, break data from every pre-order node in the left half and right half.
Continue doing so till one node, one element, is remaining.



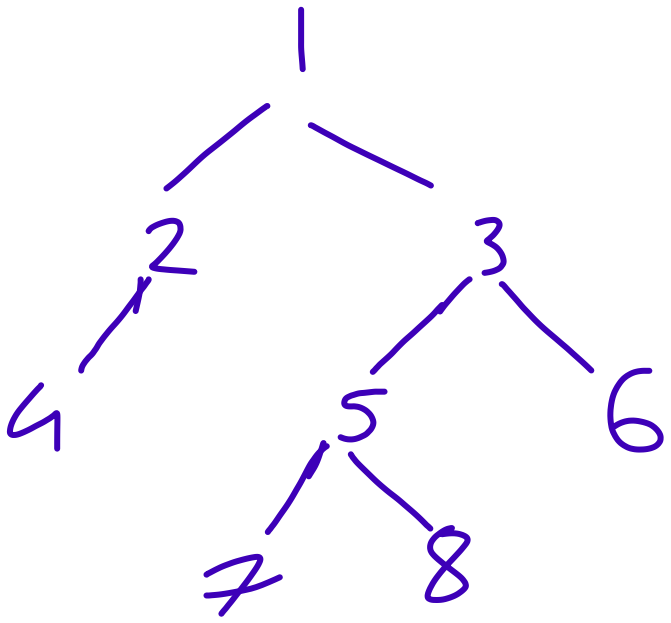
Construct tree

- preorder [3,9,20,15,7]
- inorder = [9,3,15,20,7]



Construct tree

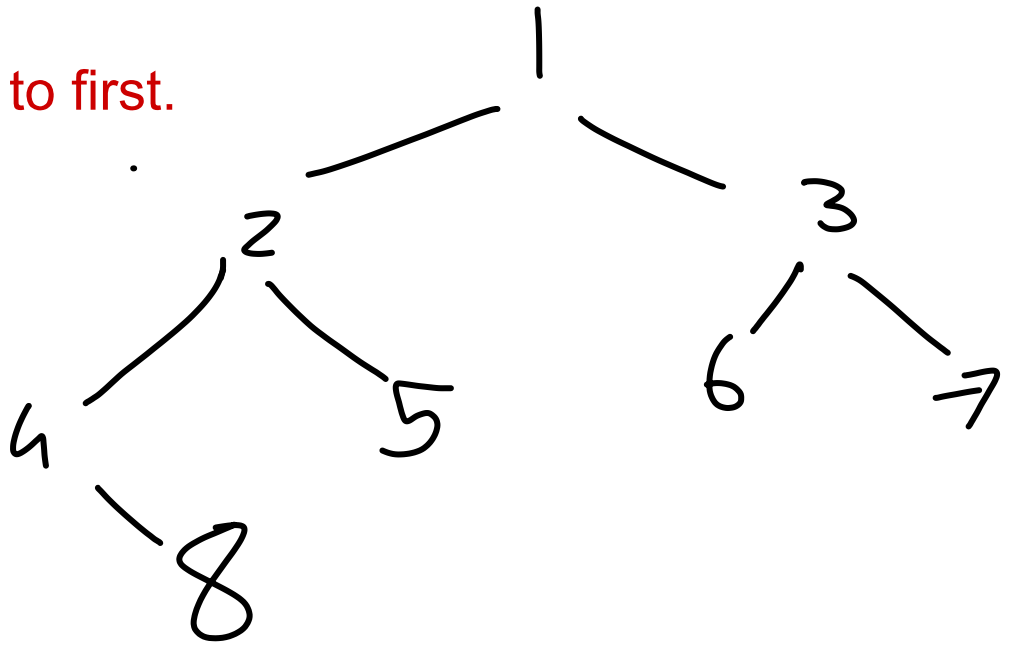
- Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }
- Preorder Traversal: { 1, 2, 4, 3, 5, 7, 8, 6 }



Construct tree

- in[] = {4, 8, 2, 5, 1, 6, 3, 7}
- post[] = {8, 4, 5, 2, 6, 7, 3, 1}

Only follow post-order
breaking sequence from last to first.

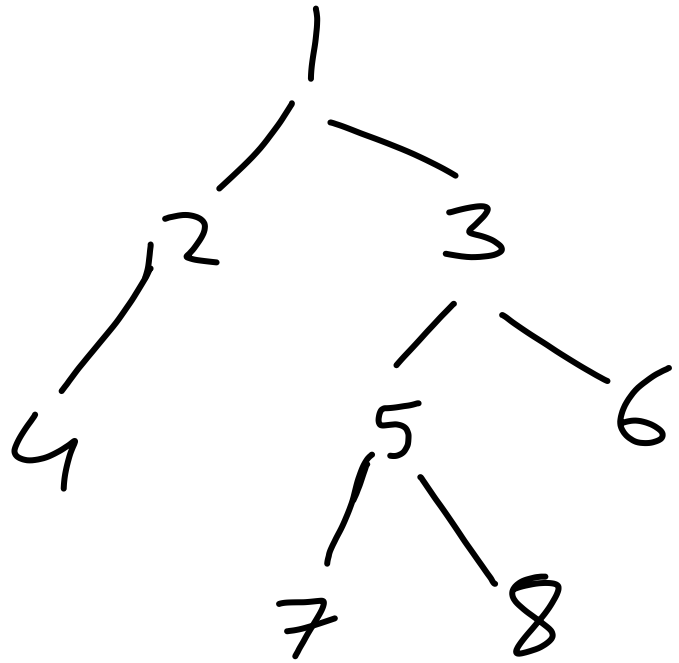


Construct tree

- inorder = [9,3,15,20,7]
- postorder = [9,15,7,20,3]

Construct tree

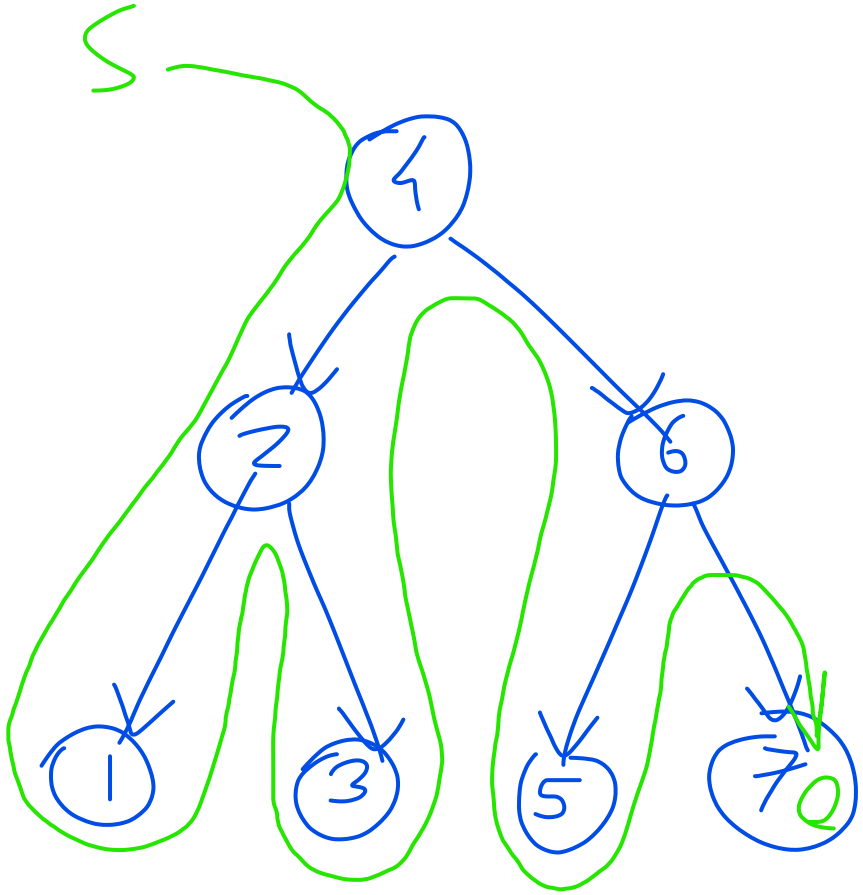
- Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }
  Postorder Traversal : { 4, 2, 7, 8, 5, 6, 3, 1 }



Threaded Tree

Specialized version of a binary search tree or a binary tree enhanced for sequential access in order without jumps

- Inorder traversal of a Binary tree can either be done using recursion or with the use of a auxiliary stack. The idea of threaded binary trees is to make inorder traversal faster and do it without stack and without recursion. A binary tree is made threaded by making all right child pointers that would normally be NULL point to the inorder successor of the node (if it exists).



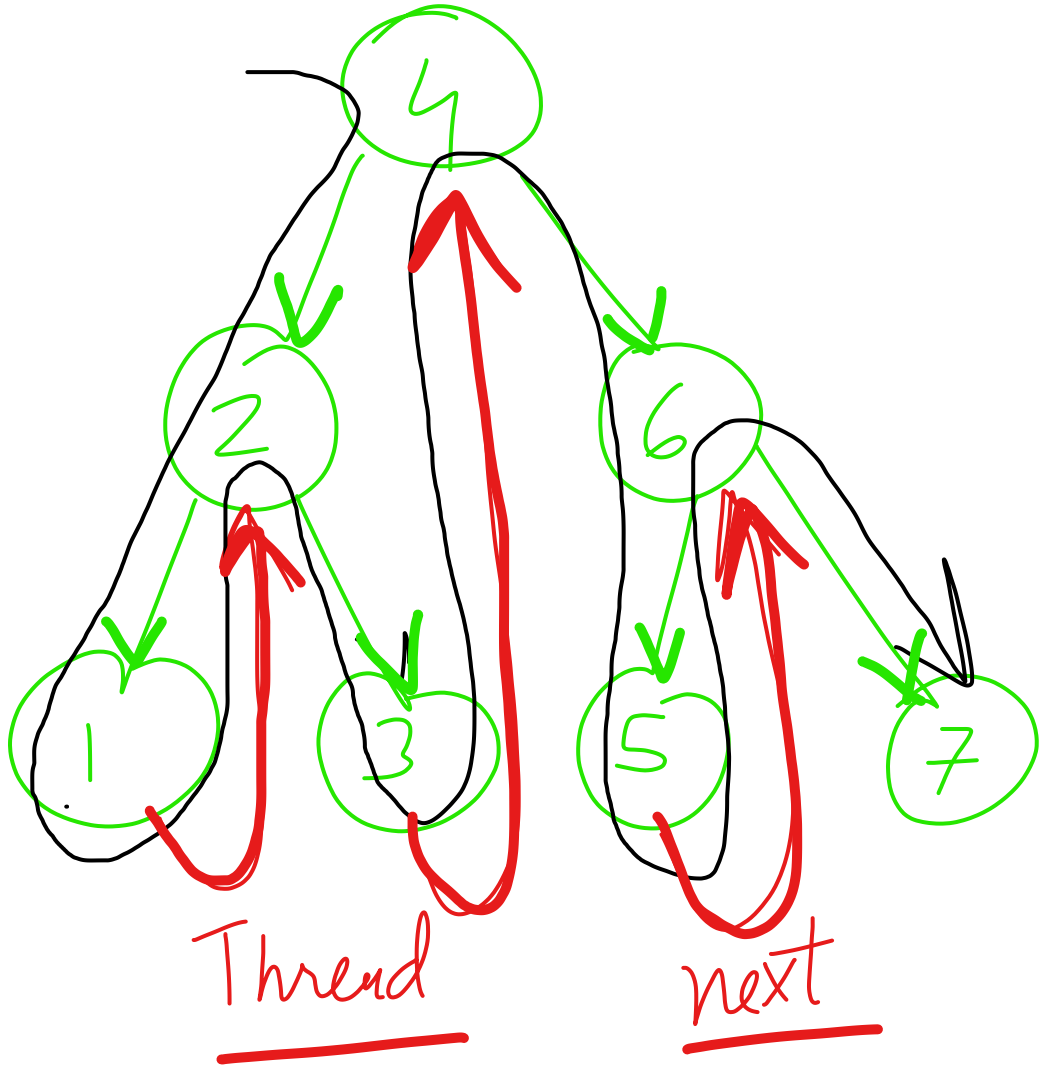
Information is needed in sequence because the property of information is only understood when it is given in proper sequence. Hence order is important.

Threaded Tree

- Single Threaded: Where a NULL right pointers is made to point to the inorder successor (if successor exists)

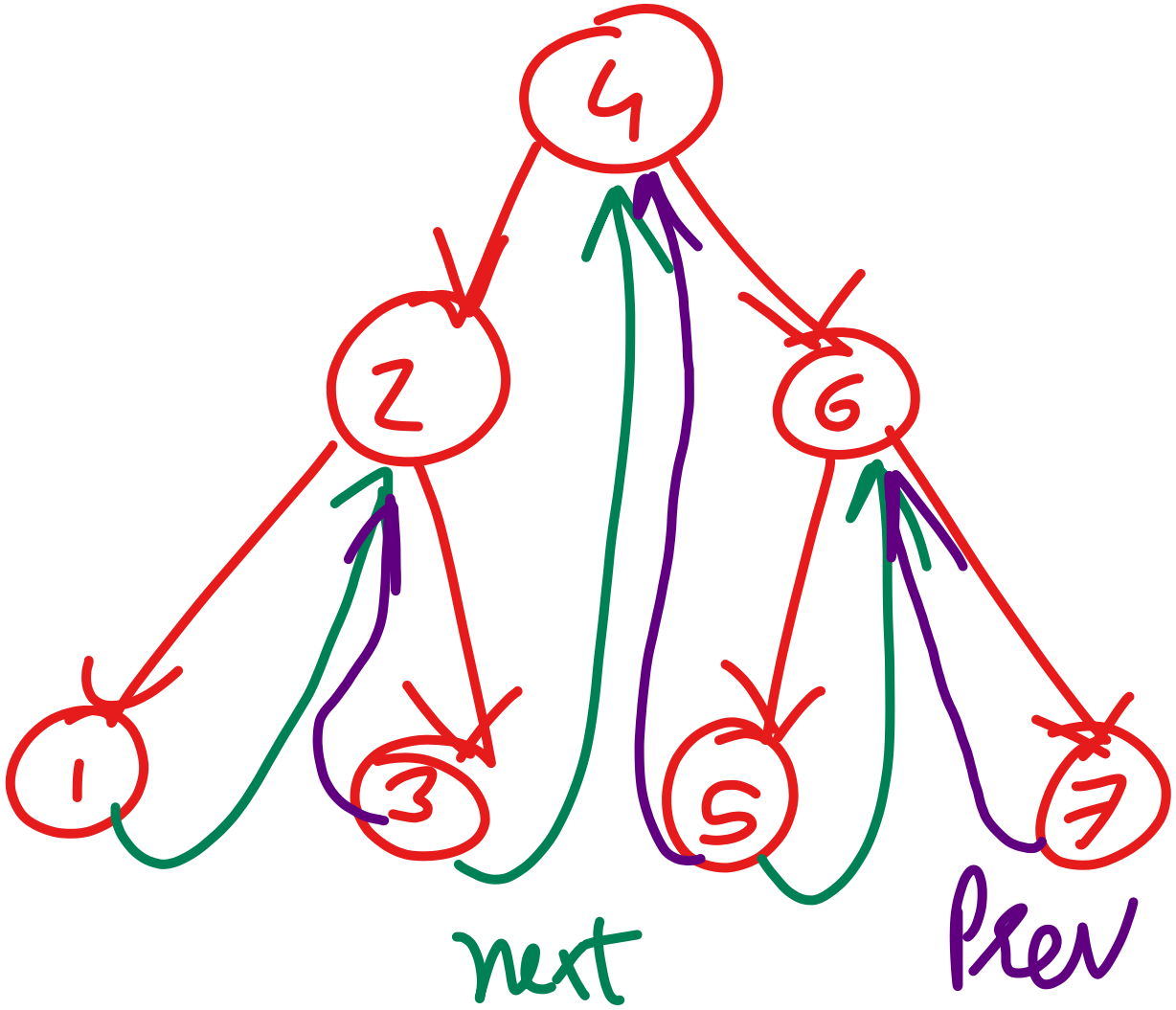
Specially enhanced to have direct access to the in-order successor so access to information becomes faster and simpler.





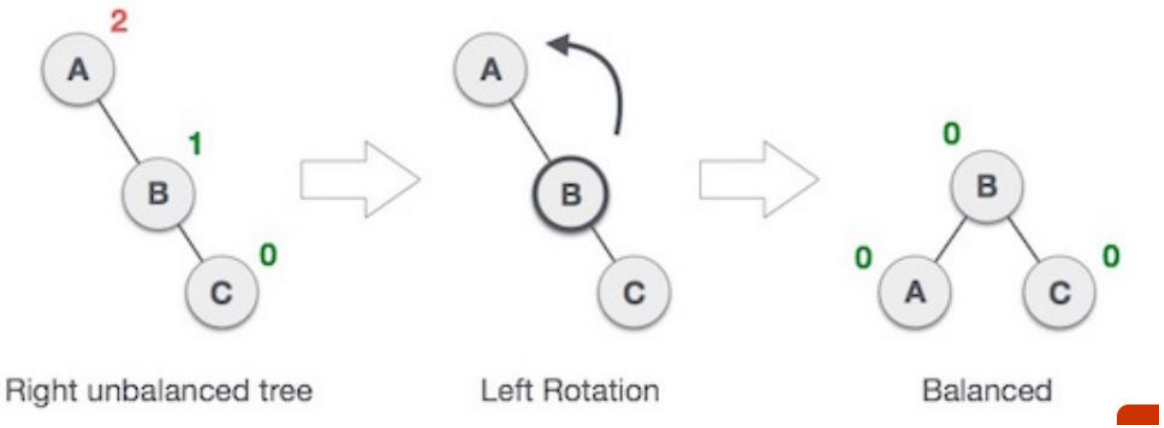
Threaded Tree

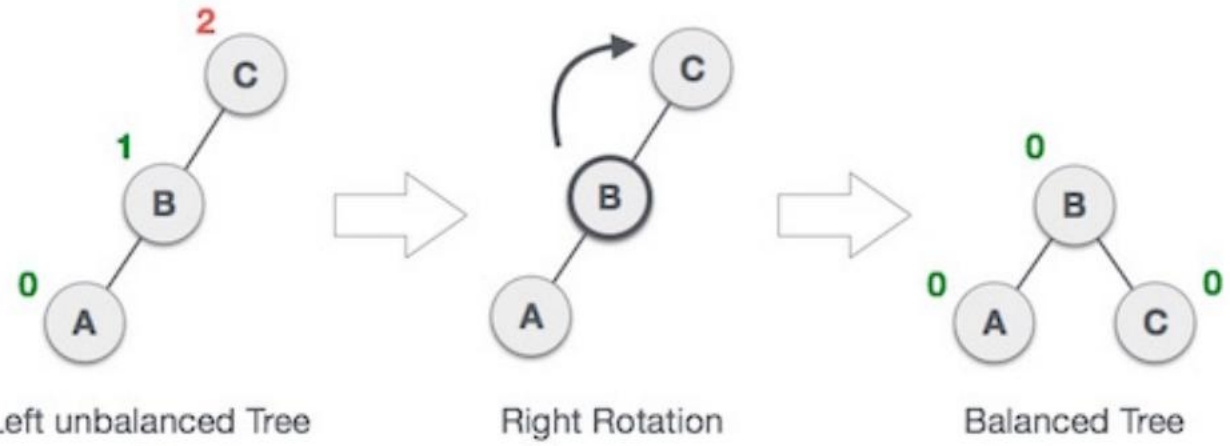
- Double Threaded: Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively. The predecessor threads are useful for reverse inorder traversal and postorder traversal.

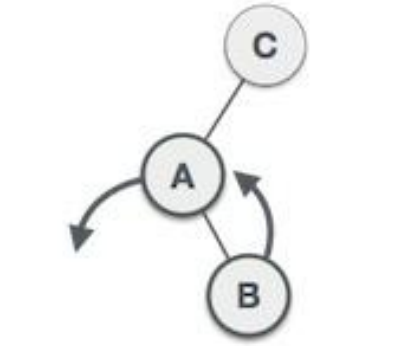


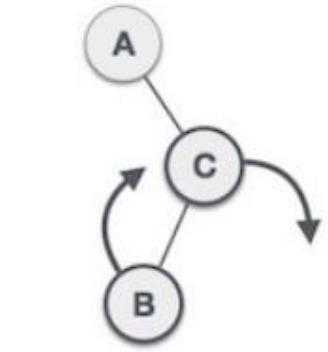
AVL TREE

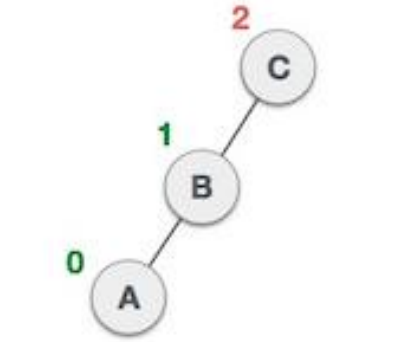
Rotations

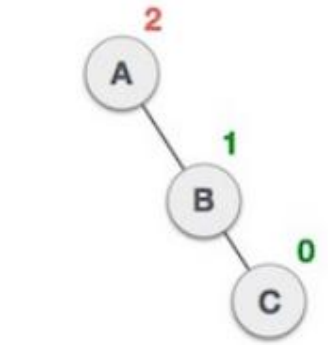

2
A
1
B
0
C
A
B
C
0
B
0
A
C
Right unbalanced tree
Left Rotation
Balanced

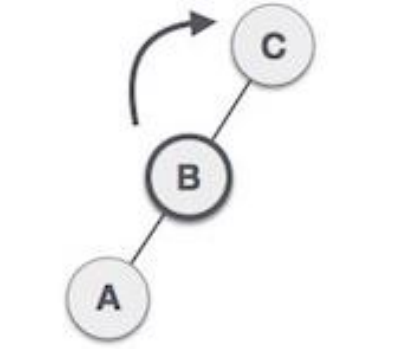

2
C
1
B
0
A
C
B
A
0
B
0
A
C
Left unbalanced Tree
Right Rotation
Balanced Tree

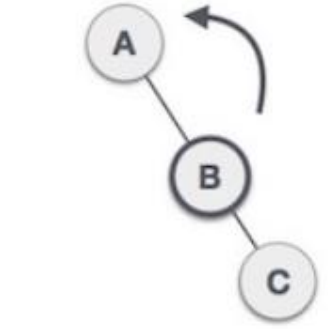

C
A
B

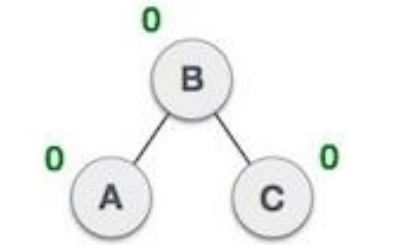

A
C
B

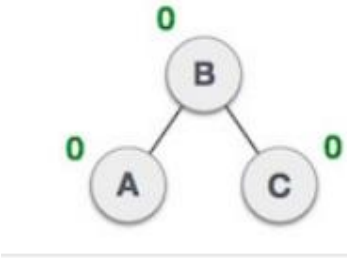

2
C
1
B
0
A


2
A
1
B
0
C


C
B
A


A
B
C


0
B
0
A
C
0


0
B
0
A
C
0

B-Tree

- B-Tree is a self-balancing search tree.
- Single node consist multiple elements this reduces number of read write and memory swaps.
- the B-Tree node size is kept equal to the disk block size
- All leaves are at the same level.
- B-Tree is defined by the term minimum degree ‘M‘.
- M is odd.
- On Order M:
  - Maximum M-1 elements at a node
  - Minimum M/2 elements at a node
  - Split when M elements reached
- All keys of a node are sorted in increasing order.
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree.
- Like other balanced Binary Search Trees, the time complexity to search, insert and delete is O(log n).
- Insertion of a Node in B-Tree happens only at Leaf Node in order and BST form.

B+ Tree

BST IMPLEMENTATION

- Static Array
- Dynamic Linked List
- Dynamic Tree node